ECS 408/608 : Operating System

Assignment 4: Multithreading

Name: Rudra Baunk Roll No.: 22268

Problem 1

Problem Statement: Write a program to create a thread T1. The main process passes two numbers to T1. In result, T1 returns the sum to the parent process for printing.

Code

```c
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>

void *compute_sum(void *arg) {
    int *nums = (int *)arg;
    int *res = malloc(sizeof(int));
    *res = nums[0] + nums[1];
    pthread_exit((void *)res);
}

int main() {
    pthread_t t1;
    int *result;

    int nums[2] = {12, 18};
    printf("Name: RUdra Baunk | Roll number: 22268\n");

    pthread_create(&t1, NULL, compute_sum, (void *)nums);
    pthread_join(t1, (void **)&result);

    printf("Sum = %d\n", *result);
    free(result);
    return 0;
}
```

Output

```
Sum = 30
```

Output Screenshot

```
rudrabaunk@LAPTOP-01SLCLOU:/mnt/c/D_drive/OS/assignment 4/ques1$ echo "NAME: RUDRA BAUNK | ROLL NUMBER: 22268"
NAME: RUDRA BAUNK | ROLL NUMBER: 22268
rudrabaunk@LAPTOP-01SLCLOU:/mnt/c/D_drive/OS/assignment 4/ques1$ gcc p1.c -o p1 -lpthread
rudrabaunk@LAPTOP-01SLCLOU:/mnt/c/D_drive/OS/assignment 4/ques1$ ./p1
Name=Rudra Baunk | Roll number = 22268
Sum = 30
rudrabaunk@LAPTOP-01SLCLOU:/mnt/c/D_drive/OS/assignment 4/ques1$
```

Figure 1: Terminal output – Problem 1

Problem 2

Problem Statement: Create a program that spawns two threads, T1 and T2. Thread T1 is responsible for generating a file named "data.txt," and T2 is tasked with writing specific content to the "data.txt" file.

Code

```c
#include <stdio.h>
#include <pthread.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int fd;

void *create_file(void *arg) {
    fd = open("data.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0)
        perror("File creation failed");
    else
        printf("File created successfully\n");
    return NULL;
}

void *write_file(void *arg) {
    char buf[128];
    int n = read(0, buf, sizeof(buf));
    write(fd, buf, n);
    printf("Data written to file\n");
    close(fd);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    printf("Name: RUdra Baunk | Roll number: 22268\n");

    pthread_create(&t1, NULL, create_file, NULL);
    pthread_join(t1, NULL);

    pthread_create(&t2, NULL, write_file, NULL);
```

```
    pthread_join(t2, NULL);

    return 0;
}
```

Output

In Terminal:

```
File created successfully
Data written to file
```

In data.txt:

```
hello ! this is my OS assignment 4
```

Output Screenshot

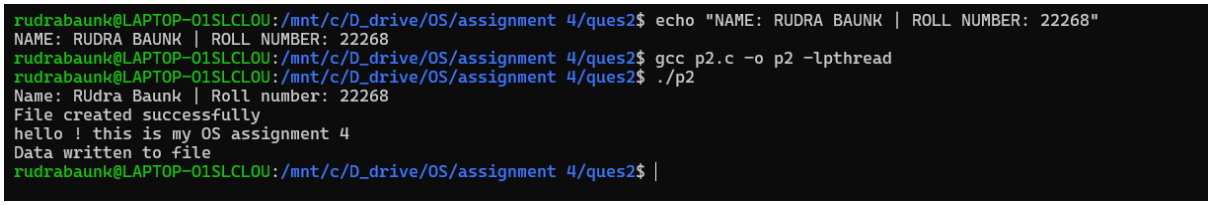

Figure 2: Terminal output – Problem 2

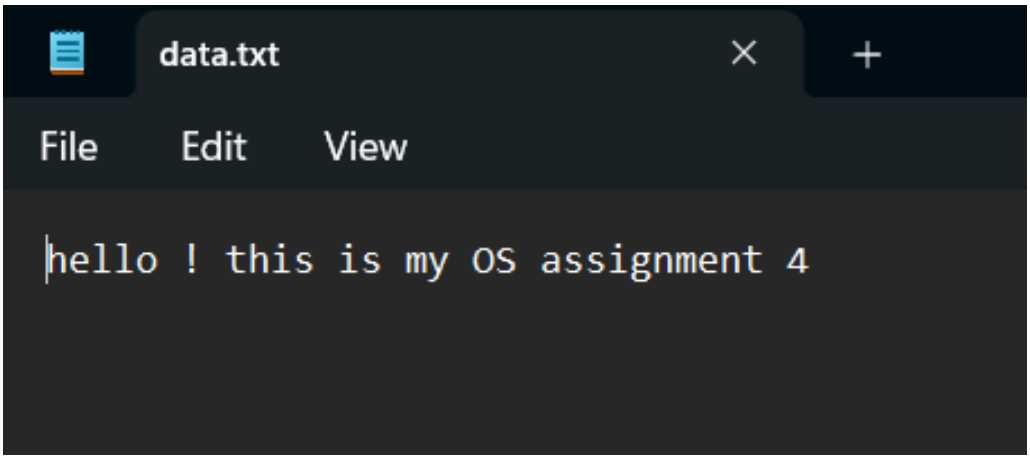

Figure 3: Terminal output – Problem 2.1

Problem 3

Problem Statement: Create a program that spawns two threads, T1 and T2. Each of the threads accepts a 10,000-element array as input. Thread T1 and T2 print each and every array element by adding an integer 2 and multiplied by 4. Both threads are cancelled after 1 second from the main thread after spawning T1 and T2.

Code

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define SIZE 10000

int arr[SIZE];
pthread_mutex_t lock;

void *process_array(void *arg) {
    pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);
    pthread_setcanceltype(PTHREAD_CANCEL_DEFERRED, NULL);

    for (int i = 0; i < SIZE; i++) {
        pthread_mutex_lock(&lock);
        printf("%d ", (arr[i] + 2) * 4);
        pthread_mutex_unlock(&lock);
        pthread_testcancel();
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    printf("Name: Rudra Baunk | Roll number: 22268\n");

    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < SIZE; i++)
        arr[i] = i;

    pthread_create(&t1, NULL, process_array, NULL);
    pthread_create(&t2, NULL, process_array, NULL);

    sleep(1);

    pthread_cancel(t1);
    pthread_cancel(t2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

```

```c
    pthread_mutex_destroy(&lock);

    printf("\nThreads cancelled after 1 second\n");
    return 0;
}
```

Output Screenshot

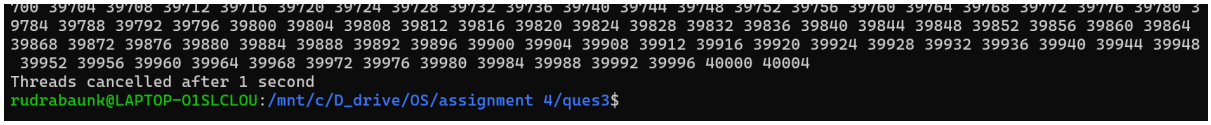

Figure 4: Terminal output – Problem 3

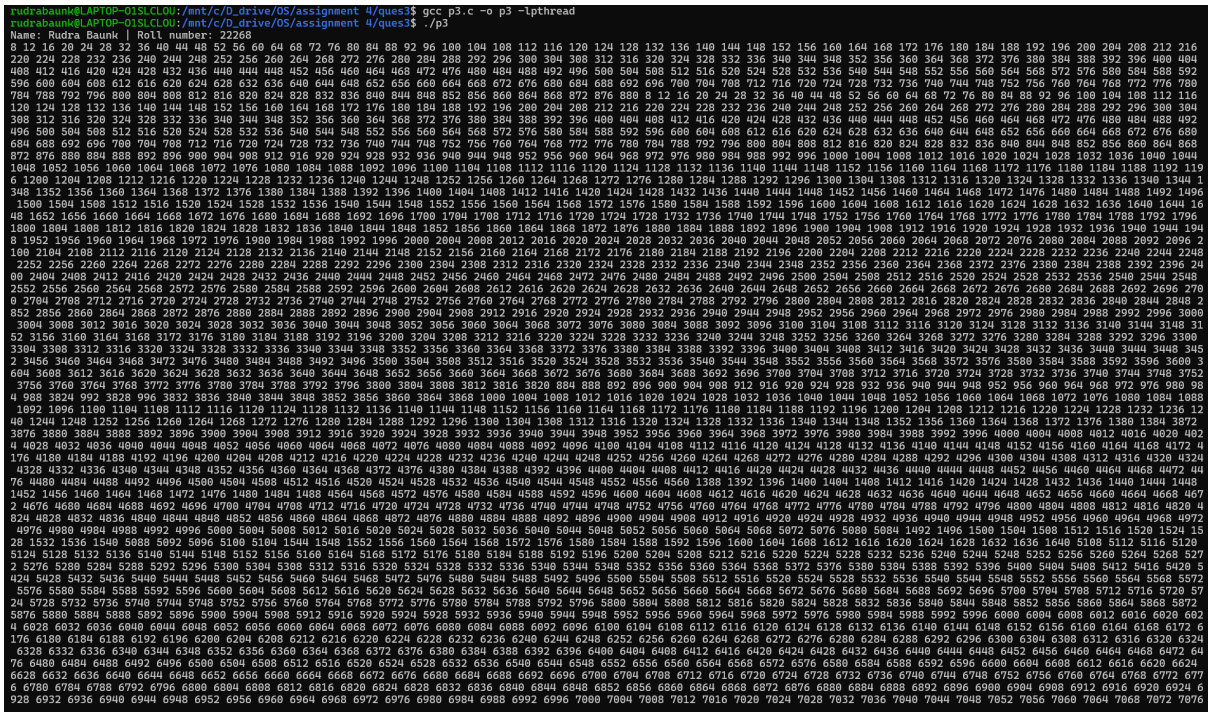

Figure 5: Terminal output – Problem 3.1